

Diseño ImgConditioning pipeline (SCA 20240326)

ImgConditioning comprende las primeras etapas del pipeline de procesamiento Hw en el que se realiza el acondicionamiento de la imagen capturada por la cámara MIPI. En este documento se definen los aspectos a tener en cuenta para configurar esta parte del pipeline.

1. ImgConditioning pipeline

El pipeline de captura de imagen consta de los siguientes bloques:

- `mipi_csi2_rx_subsys` (Xilinx IP): recibe los datos en crudo de la cámara mipi y los transforma en un stream de video (AXIStream).
- `demosaic` (Xilinx IP): recibe la imagen en crudo (Bayern pattern) y la transforma a formato RGB.
- `gamma_lut` (Xilinx IP): realiza la corrección del gamma de la imagen. Básicamente realiza una redistribución no lineal de la intensidad de la imagen para evitar la saturación en los valores más altos.
- `rgb2y` (TBD IP): realiza la conversión de formato de la imagen RGB a una imagen en niveles de gris (Y256)
- `downscale` (TBD IP): la cámara MIPI produce una imagen de un tamaño mayor al necesario por lo que es necesario reducir su tamaño a 640x480 sin perder FOV (campo de visión)

El resultado de este acondicionamiento es una imagen en niveles de gris con una profundidad de 8bpp (bitsperpixel) que se transmite en a través de un interfaz AXIStream.

2. MIPI_RX

Su configuración viene condicionada por los parámetros elegidos para el sensor MIPI (ver posibilidades del sensor elegido). Los parámetros más importantes a configurar:

- data lines: número de líneas PPI
- formato de los datos:
 - line rate
 - data type (RAW, RGG, YUV)
 - empaquetamiento de los pixels: single, dual, quad

~~Nota: en el diseño base los parámetros son 2 data lines, 280Mbps, RAW10, 1 pixel per clock,~~

Nota: en el sensor elegido no se puede cambiar la resolución de la imagen, solo se puede hacer Crop de un trozo de imagen (p. ej:640x480) esto cambia el FOV

3. Demosaic

Parámetros a configurar:

- pixels per clock
- data width
- Maximum number of columns and rows
- interpolation method
- Bayer sampling grid (“Image sensor data sheets specify whether the starting position, pixel(0,0) of the Bayer sampling grid is on a red-green or a blue-green line, and whether the first pixel is green. The Sensor Demosaic IP core supports all four possible Bayer phase combinations.”). Se configura en el registro 0x0028 del core. Revisado.
- Use UltraRAM for Line Buffers: sería conveniente utilizar este tipo de memoria para reducir el consumo de BRAM en el diseño final. La versión 9EV del chip no tiene UltraRAM

~~Nota: en el diseño base los parámetros son 1 pixel per clock, 10bits data width, 2048x1024, Fringe-Tolerant interpolation (Horizontal Zipper Artifact Removal).~~

4. Gamma

Parámetros a configurar:

- pixels per clock
- data width
- Maximum number of columns and rows. Fijar al valor de la imagen.
- LUTs de conversión para R, G y B. Configurables a través de los registros del IP

~~Nota: en el diseño base los parámetros son 1 pixel per clock, 10bits data width, 2048x1024.~~

5. RGB2Y

Este IP realiza la conversión a formato Gray de 256 niveles (Y 8bits).

- **Referencia XAPP930.pdf**

Formatos Y:

- YUV, Y:0-255
- YCrCb es la versión escalada y con offset (16) de YUV
 - o Y: 16-235
 - o Cr,Cb: 16-240

Distintas funciones de conversión:

- ITU-R BT.601 (NTSC)
 - o $Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$
 - o $Y = CA \cdot R + (1 - CA - CB) \cdot G + CB \cdot B$
 - o $CA = 0.299$; $CB = 0.114$
- ITU-R BT.709 (PAL) es:
 $Y_{709} = 0.2126 R + 0.7152 G + 0.0722 B$

- **Librería xf_cvt_color_utils de Xilinx (vitis_library 2023.2, 2022)**

https://docs.xilinx.com/r/2022.2-English/Vitis_Libraries/Utils/datamover/kernel_gen_guide.html_2_2

```
/*
 * CalculateY - calculates the Y(luma) component using R,G,B values
 * Y = (0.257 * R) + (0.504 * G) + (0.098 * B) + 16
 * An offset of 16 is added to the resultant value
 */
/*
 * CalculateGRAY - calculates the GRAY pixel value using R, G & B values
 * GRAY = 0.299*R + 0.587*G + 0.114*B
 */
```

Los pixels de entrada tienen una profundidad de 10bits por lo que el resultado final deberá truncarse (eliminando los LSB bits).

Posibilidades de implementación:

- Custom con VitisHLS. Ver nota de aplicación de Xilinx y diseño Handel-C.
- Utilizando el IP VideoProcessingSubsystem de Xilinx

a) IP custom con VitisHLS

- Utilizando la librería cvt de HLS

<https://www.hackster.io/adam-taylor/using-hls-on-an-fpga-based-image-processing-platform-8f029f>

- Utilizando código HLS

<https://www.hackster.io/news/microzed-chronicles-creating-video-streams-with-vivado-hls-79de84de7117>

cores HLS de PYNQ

<https://www.hackster.io/adam-taylor/snickerdoodle-pynq-image-processing-0b0330#toc-getting-started-1>

https://github.com/ATaylorCEngFIET/vivado_sd_image_processing-/tree/master

<https://github.com/Xilinx/PYNQ/tree/master/boards/ip/hls>

- Ver ejemplos Kria con raspi camara

<https://github.com/Xilinx/kria-vitis-platforms/blob/main/kv260/overlays/examples/smartcam/>

b) VideoProcessingSubsystem

Incluye varios tipos de ip cores, en nuestro caso solo nos interesa implementar el IP de Color Space Conversion (RGB => Y). El consumo de recursos es aprox. LUTs/ FFs / DSPs / 36k BRAM / 18k BRAM

Color Space Conversion Only RGB/YUV 444 support only demo window disabled	3349	3972	36	0	0
---	------	------	----	---	---

Parámetros a configurar:

- Color Space Conversion Only
- pixels per clock
- data width
- Max number of pixels (cols???)
- Max number of lines
- Color Space Support
- Coeficientes de la conversión (K3x3, Offset, Clamp). Configurable a través de registros del IP: **Esto es lo que define el tipo de conversión**

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} K_{11} & K_{12} & K_{13} \\ K_{21} & K_{22} & K_{23} \\ K_{31} & K_{32} & K_{33} \end{bmatrix} \begin{bmatrix} Y \\ U \\ V \end{bmatrix} + \begin{bmatrix} O_1 \\ O_2 \\ O_3 \end{bmatrix}$$

- Se incluye una API para configurar todos los registros del IP

6. Downscale

Se contemplan dos alternativas para implementar esta etapa:

- IP Resize (incluido en el ORB pipeline). Se trata de hacer una implementación no configurable por Sw que realice el downscale con el mismo método que en el pipeline ORB. Modificaciones a realizar:
 - eliminación del interfaz AXILite y configuración fija 1440x1080 => 640x480

- adecuación del interfaz AXISStream para que admita 2 pixels/clock (actualmente trabaja a 4pixels/clock)
- trabaja en el espacio GRAY

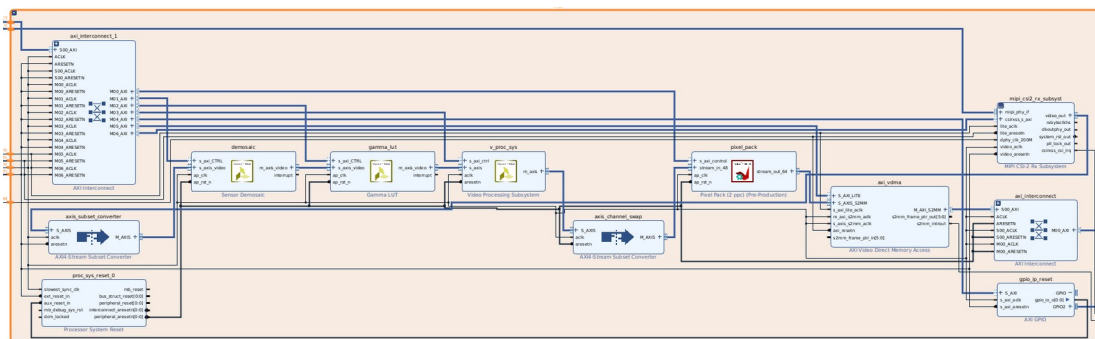
- IP VideoProcessingSubsystem. Utilizar el IP “Only Scale” que incluye también la conversión del espacio de color. En este caso es necesario configurar por software la operativa. Al ser configurable y trabajar en el espacio RGB, probablemente ocupará más recursos que el IP Resize

7. Diseños de referencia

A) Kria-PYNQ base overlay (versión 2022.2)

<https://github.com/mariodruiz/Kria-PYNQ.git>

Jerarquía mipi:



- **Cam MIPI:** genera formato de pixel RAW10, 2 pixels per clock
- **AXISStream Subset Converter:** convierte a RAW8 eliminando los 2 bits LSB de cada pixel.
Slave Interface TDATA width (bytes): 3 ; Master Interface TDATA Width (bytes): 2
TDATA Remap String: tdata[19:12],tdata[9:2]
- **Demosaic:** utiliza UltraRAM, TDATA 16b (RAW8) => TDATA 48b (2pix BGR definido por sw)

Samples per Clock	2	
Maximum Data Width	8	
Maximum Number of Columns	1920	[64 - 8192]
Maximum Number of Rows	1080	[64 - 4320]
Interpolation Method	High Resolution Interpolation	
☒ Horizontal Zipper Artifact Removal		
☒ Use UltraRAM for Line Buffers		

- **Gamma:** TDATA 48b (2pix BGR) => TDATA 48b (2pix BGR)

Samples per Clock	2	
Maximum Data Width	8	
Maximum Number of Columns	1920	[64 - 8192]
Maximum Number of Rows	1080	[64 - 4320]

- **Video Processing Subsystem:** TDATA 48b entrada BGR, salida YUV

Top Level	Color Matrix
Samples Per Clock	2
Maximum Data Width	8
Maximum Number of Pixels	1920 [64 - 8192]
Maximum Number of Lines	1080 [64 - 4320]
Video Processing Functionality	Color Space Conversion Only
Top Level Configuration Options	
Color Space Support	
☐ RGB YUV 4:4:4 YUV 4:2:2 YUV 4:2:0 ☐ RGB YUV 4:4:4 YUV 4:2:2 ☒ RGB YUV 4:4:4	

Importante: en la ventana indica los espacios de color soportados, **no la conversión**. La conversión se configura a través de los registros de conf.

Coeff. de conversión ordenados para entrada BGR

$$\text{GRAY} = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

$$K11 = 0.114$$

$$K12 = 0.587$$

$$K13 = 0.299$$

$$Y = K11 \cdot B + K12 \cdot G + K13 \cdot R$$

$$B1G1R1B0G0R0 \text{ (48b)} \Rightarrow Y1U1V1Y0V0U0 \text{ (48b)}$$

- **AXISStream Subset Converter:** reordena los canales (TDATA 48b)

TDATA Rempa String: tdata[15:0], tdata[23:16], tdata[39:24], tdata[47:40]

Y1U1V1Y0V0U0 (48b) => U0V0Y0U1V1Y1 BUG dientes de sierra al hacer intercambiar columnas de 2 en ??????

**tdata[39:24], tdata[47:40], tdata[15:0], tdata[23:16],
Y1U1V1Y0V0U0 (48b) => U1V1Y1U0V0Y0 debería ser esto**

- **Pixel_pack (2ppc): empaqueta los pixels** TDATA 48b (2pix RGB/YUV) => TDATA 64b

(dependiendo del modo el número de pixels empaquetados serán más o menos, en el caso V_8 son 8pixels de 8bits, copiando el componente Y)

Register space:

- offset: , mode

- offset: , alpha

```
// Recibe 2 pixels_per_clock de RGB/YUV (48b) envía un paquete de 64b
// en el caso V_8 son 8bits_per_pixel por tanto un paquete de 8pixels
// los canales que se empaquetan son los que están en los bits [7,0] y [31,24]
// en nuestro caso llega:
// U7V7Y7U6V6Y6 U5V5Y5U4V4Y4 U3V3Y3U2V2Y2 U1V1Y1U0V0Y0 => Y7Y6Y5Y4Y3Y2Y1Y0
```

```
#define V_8 2
```

```
case V_8:
    while (!delayed_last) {
#pragma HLS pipeline II=4
        delayed_last = last;
        bool user = false;
        ap_uint<64> data;
        for (int i = 0; i < 4; ++i) { //lee 4 veces
            if (!last) {
                stream_in_48.read(in_pixel); // lee 2 pixels
                user |= in_pixel.user;
                last = in_pixel.last;
                data(i*16 + 7, i * 16) = in_pixel.data(7,0); // copia Y1
                data(i*16 + 15, i * 16 + 8) = in_pixel.data(31,24); // copia Y2
            }
        }
        if (!delayed_last) {
            out_pixel.user = user;
            out_pixel.last = last;
            out_pixel.data = data;
            stream_out_64.write(out_pixel);
        }
    }
}
```

```

    }
}
break;

```

[PYNQ/boards/ip/hls/pixel_pack_2/pixel_pack.cpp](https://github.com/Xilinx/PYNQ/tree/a056b8455f80a145839177102288e1f1d2b8ebe3/pynq/lib)

- pixel_pack.cpp 24b => 32b
- pixel_pack_2.cpp 48 (2 pixels)=> 64b
- **AXI VDMA: 4 framebuffers**

Basic **Advanced**

Address Width (32-64) 32 bits

Frame Buffers 4

☒ Enable Write Channel ☐ Enable Read Channel

Write Channel Settings:

- Memory Map Data Width: 128
- Write Burst Size: 256
- Stream Data Width (Auto): 64
- Line Buffer Depth: 2048

Read Channel Settings:

- Memory Map Data Width: 64
- Read Burst Size: 8
- Stream Data Width: 32
- Line Buffer Depth: 512

B) Análisis de las librerías PYNQ para este diseño

<https://github.com/Xilinx/PYNQ/tree/a056b8455f80a145839177102288e1f1d2b8ebe3/pynq/lib>

STFleming and skalade Formatting changes

Name	Last commit message
..	
_pynq	Formatting changes
arduino	Formatting changes
logictools	Formatting changes
pmod	Formatting changes
pybind11	Formatting changes
pynqmicroblaze	Formatting changes
rpi	Formatting changes
tests	Formatting changes
video	Formatting changes

video

```

|--- common.py: wrappers para guardar la info de configuración
|   |--- VideoMode (height, width, stride, bits_per_pixel, bytes_per_pixel, shape)
|   |--- PixelFormat (bits_per_pixel, in_color, out_color)
|   |--- Mats de transformación de color: COLOR_OUT_GRAY ...
|   |--- genera los wrappers que contienen la información de configuración
pipeline: PIXEL_RGB, PIXEL_YCBCR, PIXEL_GRAY... los utiliza hierarchies para configurar el pipe

|--- dma.py
|   |--- AxiVDMA: driver de VDMA
|--- hierarchies.py
|   |--- VideoIn: input video pipeline (color_convert_in-> pixel_pack -> axi_vdma)
|       incluye un método configure que para rellenar la estructura del driver de videoIn
|       el pixelformat está predefinido a PIXEL_BGR
|       videoIn.configure(pixelformat=PIXEL_BGR wrapper definido en common.py)

```

```

        if self._vdma.readchannel.running:
            self._vdma.readchannel.stop() # para el pipeline
        self._color.colorspace = pixelformat.in_color
        self._pixel.bits_per_pixel = pixelformat.bits_per_pixel
        self._hdmi.start()
        input_mode = self._hdmi.mode
        self._vdma.readchannel.mode = VideoMode(
            input_mode.width,
            input_mode.height,
            pixelformat.bits_per_pixel,
            input_mode.fps,
        )
|--- VideoOut: output video pipeline (axi_vdma-> pixel_unpack -> color_convert->frontend)
    Incluye el método configure para rellenar la estructura del driver VideoOut
    El pixelformat no está predefinido
|--- pcam5.py
|--- MIPIMode: Supported video modes: r1280x720_60 = 0 ; r1920x1080_30 = 1
|--- Pcam5C: inicializa, configura el pipeline, start, stop
    UTILIZA UNA LIBRERÍA DINÁMICA COMPILADA DONDE ESTÁ EL DRIVER Del PIPELINE
    pcam5c_lib = pcam5c_ffi.dlopen(os.path.join(LIB_SEARCH_PATH, "libpcam5c.so")
    configura el mode del pixel_pack (está en _pynq)

```

_pynq/_pcam5c/pcam_mipi.c Modo de captura, virtaddr de gpio_reset, v_proc_sys, gamma_lut y demosaic

```

|--- pipeline.py
    incluye dos clases para configurar los IP (processing subsystem y
pixel_packer)
|--- ColorConverter: driver para el Colorconverter
    incluye el procedimiento colorspace(self, color) que escribe los registros del IP
@colorspace.setter
    def colorspace(self, color):
        if len(color) != 12:
            raise ValueError("Wrong number of elements in color specification")
        for i, c in enumerate(color):
            self.write(0x10 + 8 * i, int(c * 256))

|--- PixelPacker: driver
    incluye el procedimiento colorspace(self, color) que escribe los registros del IP
@bits_per_pixel.setter
    def bits_per_pixel(self, value):
        if value == 24:
            mode = 0
        elif value == 32:
            mode = 1
        elif value == 8:
            mode = 2
        elif value == 16:
            if self._resample:
                mode = 4
            else:
                mode = 3
        else:
            raise ValueError("Bits per pixel must be 8, 16, 24 or 32")
        self._bpp = value
        self.write(0x10, mode)

```

_pynq --- _pcam5c

```

|--- pcam_5c.c
    escribe los regs de la cámara con la info de las estructuras .h
    StartPcam, init_pcam
|--- pcam_5c.h
    define las estructuras (cfg_init_, cfg_1080p_30fps_) con la info de
    configuración de los registros de la cámara
|--- pcam_mipi.c
    InitImageProcessingPipe inicializa los cores Gamma, Demosaic,
    ColorConversion con el alto y ancho de la imagen, y activan el bit
    ap_start (el resto de registros se escriben a valores "dummy")

```

Video está compuesto por dos grandes bloques:

- PCAM5: mipi_rx => gamma => demosaic
- PIPELINE: color_convert (v_proc_sys) => pixel_pack

Uso y configuración de overlays

https://pynq.readthedocs.io/en/latest/overlay_design_methodology/overlay_tutorial.html

La información relativa al overlay se encuentra en 3 archivos:

- .bit
- .hwh incluye los ips y jerarquías presentes en el diseño
- .dbto permite cargar de forma dinámica un nuevo dispositivo (no incluido en el devicetree de Linux)

- La clase Overlay permite descubrir los IP y jerarquías dentro del PL, además genera instancias de **drivers genéricos** (DefaultIP) para los IPs y jerarquías.

Ejemplo:

```
# Si tenemos el IP scalar_add, para generar driver genérico acceder a sus
# registros
add_driver = overlay.scalar_add
# conociendo el offset de cada registro
add_driver.write(offset, valor)
valor_read= add_driver.read(offset)
```

- Es posible escribir un **driver custom** utilizando el driver genérico

```
class AddDriver(DefaultIP):
    def __init__(self, description):
        super().__init__(description=description)
```

```
bindto = ['xilinx.com:hls:add:1.0']
```

```
def add(self, a, b):
    self.write(0x10, a)
    self.write(0x18, b)
    return self.read(0x20)
```

```
#uso
overlay.add_driver.add(15, 20)
```

- En una jerarquía que incluya 2 IPs (p.ej: const_multiply= multiply + dma) se pueden obtener los driver de cada IP, de la siguiente forma

```
dma_driver = overlay.const_multiply.dma
multiply_driver = overlay.const_multiply.multiply
```

8. Análisis de la funcionalidad Color Space Conversion

Jupyter notebook `mipi_to_display.ipynb`

```
from kv260 import BaseOverlay
from pynq.lib.video import *
```

```
base = BaseOverlay("/home/root/jupyter_notebooks/kv260/video/base.bit")
help(base) # ofrece la información del overlay
```


IP Blocks

```
mipi/mipi_csi2_rx_subsys : pynq.lib.video.mipi_rx.MipiRx
mipi/v_proc_sys         : pynq.overlay.DefaultIP
axi_iic                  : pynq.lib.iic.AxiIIC
mipi/demosaic           : pynq.overlay.DefaultIP
mipi/gamma_lut           : pynq.overlay.DefaultIP
mipi/gpio_ip_reset      : pynq.lib.axigpio.AxiGPIO
mipi/pixel_pack          : pynq.lib.video.pipeline.PixelPacker
axi_intc                 : pynq.overlay.DefaultIP
mipi/axi_vdma            : pynq.lib.video.dma.AxiVDMA
shutdown_LPD            : pynq.overlay.DefaultIP
shutdown_HP1            : pynq.overlay.DefaultIP
zynq_ultra_ps_e_0       : pynq.overlay.DefaultIP
```

Hierarchies

```
iop_pmod0 : pynq.lib.pynqmicroblaze.pynqmicroblaze.MicroblazeHierarchy
mipi      : pynq.lib.video.pcam5c.Pcam5C
```

Interrupts

None

GPIO Outputs

None

Memories

```
iop_pmod0mb_bram_ctrl : Memory
PSDDR                  : Memory
```

Incluye 2 jerarquías. La que interesa es mipi que incluye varios IPs

ejemplo de configuración de gamma

obtengo el driver del IP

```
gamma = base.mipi.gamma_lut
width_read= gamma.read(0x10) # leo el registro width
height_read= gamma.read(0x18) # leo el registro height
print(width_read,"x",height_read)
new_width= 1280
gamma.write(0x10, new_width)
```

En este overlay hay escrito un driver para manejar el pipeline, y que permite configurar los IPs de forma automática con sólo pasar como parámetros: alto, ancho, bits_pixel

Incluye 2 jerarquías. La que interesa es mipi que incluye varios IPs

La jerarquía mipi pertenece a la clase definida en pcam5c.py

```
videomode = VideoMode(1280, 720, 24) #definida en common.py, devuelve estructura
```

ver los métodos en pcam5c.py

```
mipi.stop()
def stop(self):
    """Stop the pipeline"""
    self._vdma.readchannel.stop()
mipi.configure(videomode) # solo toca los IP: vdma y pixel_pack
# solo cambia el modo de empaquetar los pixels pero no la conversión de color
```

```

def configure(self, videomode):
    if self._vdma.readchannel.running:
        self._vdma.readchannel.stop()
    self.pixel_pack_bits_per_pixel = videomode.bits_per_pixel
    self._vdma.readchannel.mode = videomode
    return self._closecontextmanager()
mipi.start()
def start(self):
    """Start the pipeline"""
    self._vdma.readchannel.start()
    return self._stopcontextmanager()

```

He comprobado que las siguientes imágenes son iguales:

- capturando en videomode = VideoMode(1280, 720, 24), captura frame de tres canales BGR, y mostrando la imagen B:B:B
- capturar en modo videomode = VideoMode(1280, 720, 8), captura frame de 1 canal (B)

es decir, como no se ha tocado la conversión de color, las imágenes están en modo BGR y cuando empaqueto con pixels de 8bits se queda con la componente B de cada pixel.

Es necesario cambiar la conversión de color a YUV y empaquetar con 8bits por pixel para que genere un frame de un solo canal (Y)

Para cambiar el espacio de color parece que no hay una función predefinida, por lo que hay que escribir en los registros del IP directamente. He reutilizado el código de **lib/video/pipeline.py**

```

def colorspace(color):
    if len(color) != 12:
        raise ValueError("Wrong number of elements in color specification")
    for i, c in enumerate(color):
        vproc.write(0x10 + 8 * i, int(c * 256))

def configBGR2YCBCR():
    colorspace(COLOR_IN_YCBCR)

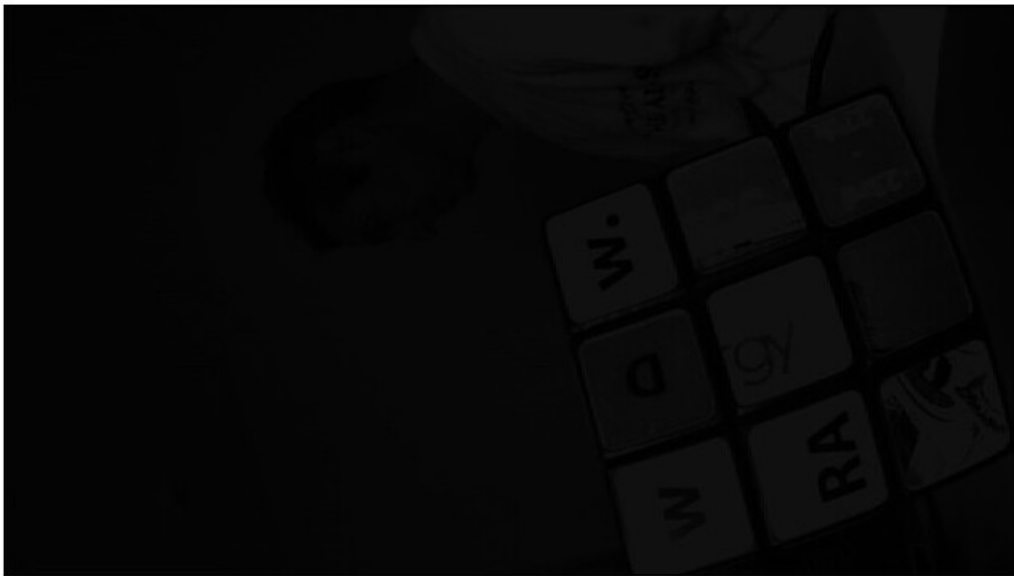
COLOR_IN_YCBCR = [0.114, 0.587, 0.299,
0.5, -0.331264, -0.168736,
-0.081312, -0.41866, 0.5,
0, 0.5, 0.5]

videomode = VideoMode(1280, 720, 8) #pixel packer in V8 mode to store just the Y channel
mipi.stop()
mipi.configure(videomode)
configBGR2YCBCR()
mipi.start()
frame = mipi.readframe()
PIL.Image.fromarray(frame)

```

La imagen obtenida es muy oscura. Parece que el factor de escala utilizado para los parámetros (256) es incorrecto.

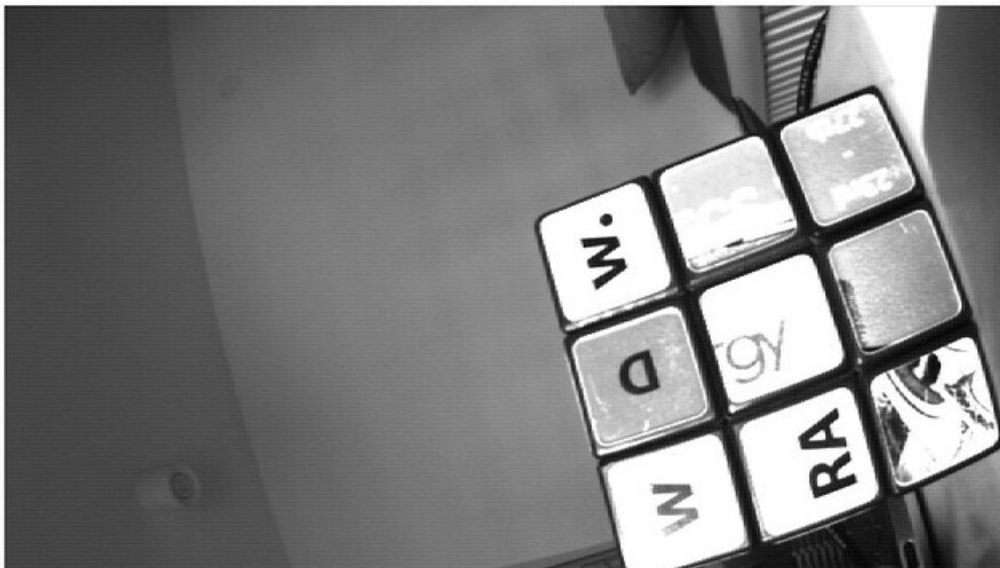
Out[70]:



He cambiado el factor de escala a 4096 (pero dejando los offset con escala 256), y parece que la imagen mejora (aunque los blancos están algo saturados)

```
def colorspace(color):
    if len(color) != 12:
        raise ValueError("Wrong number of elements in color specification")
    for i, c in enumerate(color):
        vproc.write(0x50 + 8 * i, int(c * 4096))
    vproc.write(0x98, 0) # write the offsets
    vproc.write(0x0a0, 128) # 0.5*256
    vproc.write(0x0a8, 128) # 0.5*256
```

Out[78]:



Según el **VideoProcessingSubsystemProductGuide PG231 2022**

These are RGB to YUV equations:

$$Y = (306 * R + 601 * G + 117 * B) >> 10$$

$$U = (1 << (HSC_BITS_PER_COMPONENT - 1)) + (((B - Y) * 504) >> 10)$$

$$V = (1 << (HSC_BITS_PER_COMPONENT - 1)) + (((R - Y) * 898) >> 10)$$

Note: 1. HSC_BITS_PER_COMPONENT = C_MAX_DATA_WIDTH (here C_MAX_DATA_WIDTH can be set by user through the GUI.)

Esto corresponde a un factor de escala de **1024 con la matriz de conversión COLOR_IN_YCBCR**

Prueba comparando distintos factores de escala y offsets

videomode = VideoMode(1280, 720, 24)

Conf original BGR bypass sin colorconversion

```
In [5]: #reorganizamos para que la imagen sea correcta  
PIL.Image.fromarray(frame[:, :, [2, 1, 0]])  
#frame.shape
```

Out[5]:



Sólo el canal B

```
In [6]: # Mostramos sólo el canal B  
PIL.Image.fromarray(frame[:, :, [0, 0, 0]])
```

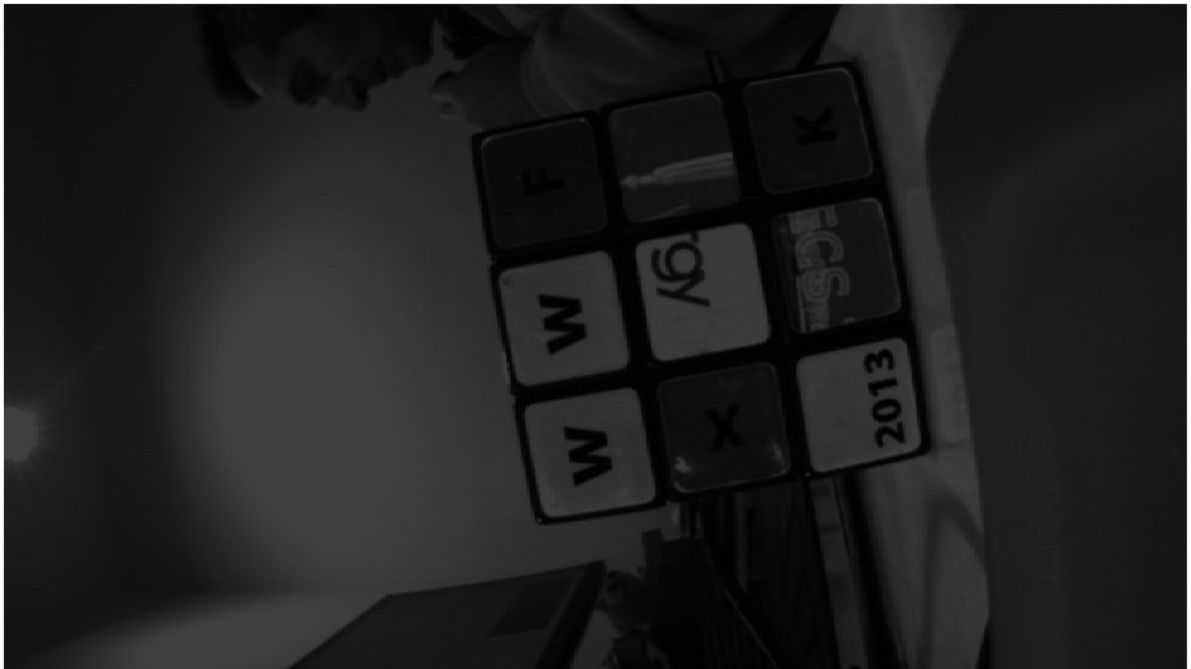
Out[6]:



ColorConversion BGR2YCRCB
factor de escala 1024 offsets 256

```
In [10]: frame = mipi.readframe()  
         PIL.Image.fromarray(frame)
```

Out[10]:



factor de escala 2048 offsets 0

```
In [20]: frame = mipi.readframe()  
         PIL.Image.fromarray(frame)
```

Out[20]:



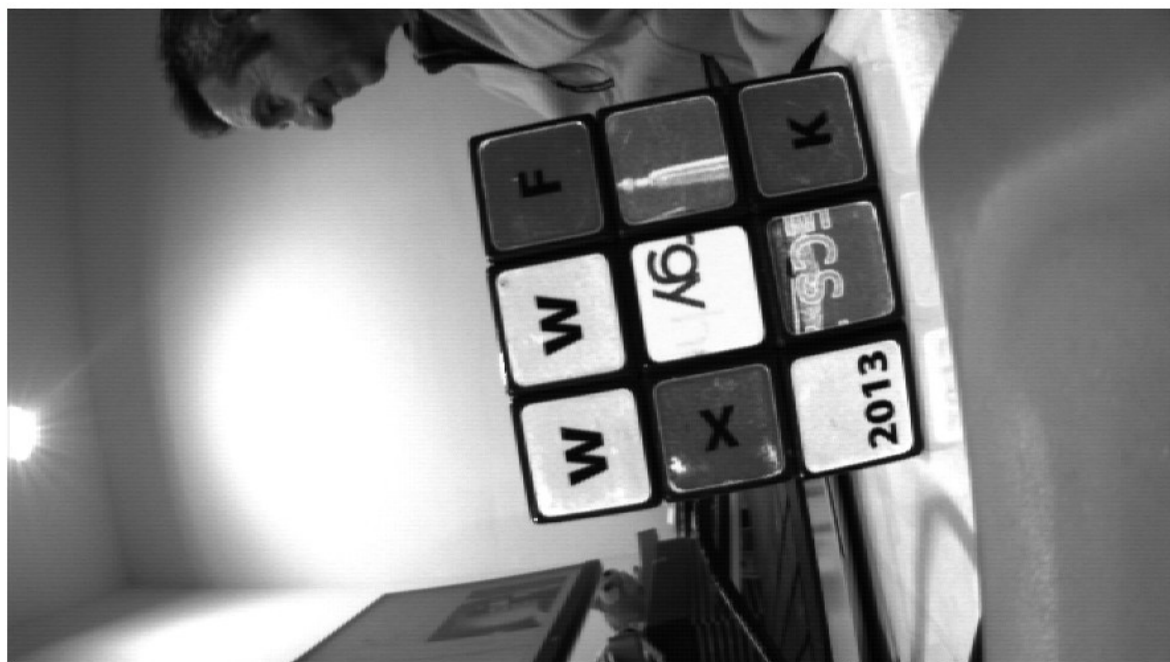
factor de escala 2048 offsets 128

Out[39]:



factor de escala 4096 offsets 0

Out[63]:



factor de escala 4096 offsets 128

Out[43]:



factor de escala 4096 offsets 256

Out[48]:



factor de escala 4096 offsets 512

Out[53]:



Nota: parece que los offsets tengan asignados un número de bits = 8, ya que cuando escribimos cualquier valor > 512 , se leen 0 en sus registros

Análisis de coeficientes para la conversión

https://support.xilinx.com/s/question/0D54U00007K2i8NSAR/vpss-csc-demo-window-bug?language=en_US

El driver proporcionado por Xilinx para manejar el V_proc_sys IP incluye 2 erratas (a tener en cuenta si utilizamos estos drivers):

- Aunque la ventana demo (window demo) esté deshabilitada en el IP, se intenta configurar los coef de esta ventana. Esto sobre escribe los coef. de la conversión con valores no inicializados.
- Los coeficientes para la conversión BGR2Y contienen una errata. El coef. es erróneo: `RGB2YCC[0][2] = (s32) ( 0.144*(float)scale_factor); //K13`
Debería ser 0.114 en lugar de 0.144.

En el driver se puede observar el factor de escala utilizado para enviar los coeficientes al IP.
xv_csc_l2.c

```
#define XV_CSC_COEFF_FRACTIONAL_BITS    (12)
s32 scale_factor = (1<<XV_CSC_COEFF_FRACTIONAL_BITS);
```

los coeficientes definidos en el driver para BT_601 son:

```
s32 scale_factor = (1<<XV_CSC_COEFF_FRACTIONAL_BITS);
s32 bpcScale = (1<<(pixPrec-8)); //pixPrec, es InstancePtr->ColorDepth
switch(cstdOut)
{
    case XVIDC_BT_601:
        switch(cRangeOut)
        {
            case XVIDC_CR_0_255:
                RGB2YCC[0][0] = (s32) ( 0.2568*(float)scale_factor); //K11
                RGB2YCC[0][1] = (s32) ( 0.5041*(float)scale_factor); //K12
                RGB2YCC[0][2] = (s32) ( 0.0979*(float)scale_factor); //K13
                RGB2YCC[1][0] = (s32) (-0.1482*(float)scale_factor); //K21
```



```

    RGB2YCC[1][1] = (s32) (-0.2910*(float)scale_factor); //K22
    RGB2YCC[1][2] = (s32) ( 0.4393*(float)scale_factor); //K23
    RGB2YCC[2][0] = (s32) ( 0.4393*(float)scale_factor); //K31
    RGB2YCC[2][1] = (s32) (-0.3678*(float)scale_factor); //K32
    RGB2YCC[2][2] = (s32) (-0.0714*(float)scale_factor); //K33
    RGB2YCC[0][3] = 16*bpcScale; //R Offset
    RGB2YCC[1][3] = 128*bpcScale; //G Offset
    RGB2YCC[2][3] = 128*bpcScale; //B Offset
    break;

case XVIDC_CR_16_235:
    RGB2YCC[0][0] = (s32) ( 0.299*(float)scale_factor); //K11
    RGB2YCC[0][1] = (s32) ( 0.587*(float)scale_factor); //K12
    RGB2YCC[0][2] = (s32) ( 0.144*(float)scale_factor); //K13 errata
    RGB2YCC[1][0] = (s32) (-0.172*(float)scale_factor); //K21
    RGB2YCC[1][1] = (s32) (-0.339*(float)scale_factor); //K22
    RGB2YCC[1][2] = (s32) ( 0.511*(float)scale_factor); //K23
    RGB2YCC[2][0] = (s32) ( 0.511*(float)scale_factor); //K31
    RGB2YCC[2][1] = (s32) (-0.428*(float)scale_factor); //K32
    RGB2YCC[2][2] = (s32) (-0.083*(float)scale_factor); //K33
    RGB2YCC[0][3] = 0*bpcScale; //R Offset
    RGB2YCC[1][3] = 128*bpcScale; //G Offset
    RGB2YCC[2][3] = 128*bpcScale; //B Offset
    break;

case XVIDC_CR_16_240:
    RGB2YCC[0][0] = (s32) ( 0.2921*(float)scale_factor); //K11
    RGB2YCC[0][1] = (s32) ( 0.5735*(float)scale_factor); //K12
    RGB2YCC[0][2] = (s32) ( 0.1113*(float)scale_factor); //K13
    RGB2YCC[1][0] = (s32) (-0.1686*(float)scale_factor); //K21
    RGB2YCC[1][1] = (s32) (-0.3310*(float)scale_factor); //K22
    RGB2YCC[1][2] = (s32) ( 0.4393*(float)scale_factor); //K23
    RGB2YCC[2][0] = (s32) ( 0.4393*(float)scale_factor); //K31
    RGB2YCC[2][1] = (s32) (-0.4184*(float)scale_factor); //K32
    RGB2YCC[2][2] = (s32) (-0.0812*(float)scale_factor); //K33
    RGB2YCC[0][3] = 0*bpcScale; //R Offset
    RGB2YCC[1][3] = 128*bpcScale; //G Offset
    RGB2YCC[2][3] = 128*bpcScale; //B Offset
    break;

```

9. Análisis de la funcionalidad Downscale

A) Configuración del IP Video Processing Subsystem para que realice las dos tareas: Color Space Conversion y Scaler:

The screenshot shows the configuration interface for the Video Processing Subsystem, specifically the 'Scaler' tab. The 'Top Level' tab is also visible. The 'Scaler' tab has a 'Top Level' sub-tab and a 'Scaler' sub-tab. The 'Scaler' sub-tab is active, showing the 'Algorithm' section with three radio buttons: 'Bilinear', 'Bicubic' (selected), and 'Polyphase'. The 'Top Level' sub-tab shows various configuration options: 'Samples Per Clock' (2), 'Maximum Data Width' (8), 'Maximum Number of Pixels' (1920), 'Maximum Number of Lines' (1080), and 'Video Processing Functionality' (Scaler Only). Under 'Top Level Configuration Options', 'Enable Color Space Conversion' is checked. The 'Color Space Support' section has three radio buttons: 'RGB | YUV 4:4:4 | YUV 4:2:2 | YUV 4:2:0', 'RGB | YUV 4:4:4 | YUV 4:2:2', and 'RGB | YUV 4:4:4' (selected).

Registros de configuración de la parte Scaler:

Vertical scaler

- 0x10 Input Height
- 0x18 Width
- 0x020 Output Height
- 0x030 Color Mode

Horizontal scaler

- 0x010 Height

- 0x018 Input Width
- 0x020 Output Width
- 0x028 Color Mode

No existe ejemplo de aplicación y en la documentación no queda claro como utilizar los registros (algunos se solapan)

B) IP Resize_scaler. Modificación del IP Resize (acSLAM).

- Versión 1: 1440x1080, n=4

Utilization Estimates					
Summary					
Name	BRAM_18K	DSP48E	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	-	0	361	-
FIFO	-	-	-	-	-
Instance	4	234	8402	32685	0
Memory	-	-	-	-	-
Multiplexer	-	-	-	1091	-
Register	-	-	858	-	-
Total	4	234	9260	34137	0
Available	288	1248	234240	117120	64
Utilization (%)	1	18	3	29	0

- Versión 2: 1440x1080, n=1

Utilization Estimates					
Summary					
Name	BRAM_18K	DSP48E	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	-	0	361	-
FIFO	-	-	-	-	-
Instance	2	78	4251	13666	0
Memory	-	-	-	-	-
Multiplexer	-	-	-	1091	-
Register	-	-	620	-	-
Total	2	78	4871	15118	0
Available	288	1248	234240	117120	64
Utilization (%)	~0	6	2	12	0

- Versión 3: 1440x1080, n=1, sin scale=1

Utilization Estimates					
Summary					
Name	BRAM_18K	DSP48E	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	-	0	346	-
FIFO	-	-	-	-	-
Instance	2	71	3802	10983	0
Memory	-	-	-	-	-
Multiplexer	-	-	-	836	-
Register	-	-	618	-	-
Total	2	71	4420	12165	0
Available	288	1248	234240	117120	64
Utilization (%)	~0	5	1	10	0

- Versión 4: 1440x1080, n=1, sin scale=1 y configuración fija

Utilization Estimates					
Summary					
Name	BRAM_18K	DSP48E	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	-	0	176	-
FIFO	-	-	-	-	-
Instance	2	70	2489	9876	0
Memory	-	-	-	-	-
Multiplexer	-	-	-	426	-
Register	-	-	340	-	-
Total	2	70	2829	10478	0
Available	288	1248	234240	117120	64
Utilization (%)	~0	5	1	8	0

scale=2.250000
width=5A0, 1440
height=438, 1080
scale=9000
inv_scale=1C71
new_width=280, 640
new_height=1E0, 480

versión	BRAM	DSP48	FF	LUT
V1: configurable, 16ppc	4	234	9260	34137
V2: configurable, 4ppc	2	78	4871	15118
V3: configurable (w/o scale=1), 4ppc	2	71	4420	12165
V4: no config, 4ppc	2	70	2829	10478

El consumo de la version no configurable es similar a la versión configurable , **por tanto escogemos la versión rsz_scalern1 (n=1 4pixels/ciclo, scale configurable excepto scale=1)**

Prueba 1: prueba en kria (acSLAM) como coprocesador

NOTA: hay que cambiar el datawidth del Read channel a 32b (4pixels)

```
# scale=2.25 => (640x480) ok
# scale=2.0 => (720x540) ok
# scale=1.5 => (960x720) ok
```

Prueba 2: prueba en pipeline de captura MIPI (Kria-PYNQ) **proyecto 2022.1 kv260/ImgCnd** esquema:

```
v_proc => Y1U1V1Y0U0V0 => Swap => U1V1Y1U0V0Y0 (48b) => yuv2gray_pack
=>Y3Y2Y1Y0 (32b) => rsz_scalern1 => Y3Y2Y1Y0 (32b) => graypixels_pack => Y7Y6Y5Y4
Y3Y2Y1Y0 (64b)
```

IP preparados en **work/kria/Kria-PYNQ/kv260/HLS/**

Sintetizar IPs en versión vivado 2022.1 aunque el proyecto es 2020.1

10. Integración de ImgConditioning en proyecto T6.2

Han aparecido problemas para recuperar la imagen GRAY cuando integramos ImgCond en el pipeline del proyecto T6.2 .

Adjunto la comparativa de configuración entre la versión Kria (correcta) y la versión Alynx donde se está haciendo la integración.

Configuración Registros Kria (leída del diseño base.bit)

```
----- Gamma Config -----
ImgSize= 1280 x 720
VideoFormat= 0
LUT_RED= 65536
LUT_GREEN= 65536
LUT_BLUE= 65536
----- Demosaic Config -----
ActivePixels= 1280 x 720
BayerPhase= 0
----- V_Proc_Sys Config -----
ImgSize= 1280 x 720
input_format= 0 output_format= 0
Coef_row1= [ 4096 , 0 , 0 ]
Coef_row2= [ 0 , 4096 , 0 ]
Coef_row3= [ 0 , 0 , 4096 ]
Offset= [ 0 , 0 , 0 ]
Clamp= [ 0 , 255 ]
```

a) MIPI CSI-2
- Kria

Component Name: mipi/mipi_csi2_nv_subsys

Configuration | Shared Logic | Pin Assignment | Application Example Design

Subsystem Options

Pixel Format: RAW10 | Serial Data Lanes: 2

☒ Include Video Format Bridge (VFB)

☐ Support CSI Spec V2_0

DPHY Options

Line Rate (Mbps): 672 | [80 - 2500]

☒ D-PHY Register Interface

☐ Enable HS and ESC Timeout Counters/Registers

CSI-2 Options

☒ CSI2 Controller Register Interface

☐ Embedded non-image Interface

☒ Filter User Defined data types

Line Buffer Depth: 4096

VFB Options

Allowed VC: All | Pixels Per Clock: 2

TUSER Width: 1

Resource improvement options

☒ Enable CRC ☐ Enable Active Lanes

- T6.2

Component Name: mipi_csi2_nv_subsys_0

Configuration | Shared Logic | Pin Assignment | Application Example Design

Subsystem Options

Pixel Format: RAW10 | Serial Data Lanes: 1

☒ Include Video Format Bridge (VFB)

☒ Support CSI Spec V2_0

☐ Support VCX Feature

DPHY Options

Line Rate (Mbps): 1188 | [80 - 2500]

☐ D-PHY Register Interface

CSI-2 Options

☒ CSI2 Controller Register Interface

☐ Embedded non-image Interface

☐ Filter User Defined data types

Line Buffer Depth: 2048

VFB Options

Allowed VC: All | Pixels Per Clock: 2

TUSER Width: 1

Resource improvement options

☒ Enable CRC ☒ Enable Active Lanes

b) AXIS_subset_converter
- Kria

Component Name: mipi/axis_subset_converter

Slave Interface Signal Properties

☒ Enable TREADY Yes

☒ TDATA Width (bytes) 3 | [0 - 512]

☒ Enable TSTRB No

☒ Enable TKEEP No

☒ Enable TLAST Yes

☒ TID Width (bits) 0 | [0 - 32]

☒ DEST Width (bits) 10 | [0 - 32]

☒ USER Width (bits) 1 | [0 - 4096]

Master Interface Signal Properties

☒ Enable TREADY Yes

☒ TDATA Width (bytes) 2 | [0 - 512]

☒ Enable TSTRB No

☒ Enable TKEEP No

☒ Enable TLAST Yes

☒ TID Width (bits) 0 | [0 - 32]

☒ DEST Width (bits) 10 | [0 - 32]

☒ USER Width (bits) 1 | [0 - 4096]

Extra Settings

TDATA Remap String: tdata[19:12],tdata[9:2]

TUSER Remap String: tuser[0:0]

TID Remap String: 1'b0

TDEST Remap String: tdest[9:0]

TKEEP Remap String: 1'b0

TSTRB Remap String: 1'b0

TLAST Remap String: tlast[0]

Generate TLAST: 0 | [0 - 256]

Enable ACLKEN: No

- T6.2

Slave Interface Signal Properties

☒ Enable TREADY Yes

☒ TDATA Width (bytes) 3 | [0 - 512]

☒ Enable TSTRB No

☒ Enable TKEEP No

☒ Enable TLAST Yes

☒ TID Width (bits) 0 | [0 - 32]

☒ DEST Width (bits) 10 | [0 - 32]

☒ USER Width (bits) 1 | [0 - 4096]

Master Interface Signal Properties

☒ Enable TREADY Yes

☒ TDATA Width (bytes) 2

☒ Enable TSTRB No

☒ Enable TKEEP No

☒ Enable TLAST Yes

☒ TID Width (bits) 0

☒ DEST Width (bits) 10

☒ USER Width (bits) 1

Extra Settings

TDATA Remap String: tdata[19:12],tdata[9:2]

TUSER Remap String: tuser[0:0]

TID Remap String: 1'b0

TDEST Remap String: tdest[9:0]

TKEEP Remap String: 1'b0

TSTRB Remap String: 1'b0

TLAST Remap String: tlast[0]

Generate TLAST: 0 | [0 - 256]

Enable ACLKEN: No

c) Demosaic (maximum Data Width => "This parameter should match the Video Component Width of the video IP core connected to the slave AXI4-Stream video interface")

- Kria (UltraRAM, datawidth)

- T6.2

Component Name: mipi/demosaic

Samples per Clock: 2

Maximum Data Width: 8

Maximum Number of Columns: 1920 | [64 - 8192]

Maximum Number of Rows: 1080 | [64 - 4320]

Interpolation Method: High Resolution Interpolation

☒ Horizontal Zipper Artifact Removal

☒ Use UltraRAM for Line Buffers

Component Name: v_demosaic_0

Samples per Clock: 2

Maximum Data Width: 10

Maximum Number of Columns: 2048 | [64 - 8192]

Maximum Number of Rows: 1100 | [64 - 4320]

Interpolation Method: High Resolution Interpolation

☒ Horizontal Zipper Artifact Removal

☐ Use UltraRAM for Line Buffers

d) Gamma (maximum Data Width => "This parameter should match the Video Component Width of the video IP core connected to the slave AXI4-Stream video interface")

- Kria

- T6.2

Component Name: mipi/gamma Lut

Samples per Clock: 2

Maximum Data Width: 8

Maximum Number of Columns: 1920 | [64 - 8192]

Maximum Number of Rows: 1080 | [64 - 4320]

Component Name: v_gamma_lut_0

Samples per Clock: 2

Maximum Data Width: 10

Maximum Number of Columns: 2048 | [64 - 8192]

Maximum Number of Rows: 1100 | [64 - 4320]

Problema encontrado: Inversión de los niveles extremos de gris (los blancos se vuelven grisáceos y los negros se aclaran)

Solución: Generar los valores de gamma teniendo en cuenta el nuevo datawidth (8bits)

e) V_proc_SS (LEER TODOS LOS REGITROS KRIA)

BUG relacionado con demo window, si se utilizan los drivers de Xilinx:

https://support.xilinx.com/s/question/0D54U00007K2i8NSAR/vpss-csc-demo-window-bug?language=en_US

Solución: disable demo window + modificar el driver “XV_CscSetColorspace() to remove the last call to cscUpdateIPReg()”

```
//write IP Registers
```

```
cscUpdateIPReg(InstancePtr, UPDT_REG_FULL_FRAME);
```

```
cscUpdateIPReg(InstancePtr, UPD_REG_DEMO_WIN); // eliminar
```

- Kria

Top Level Configuration Options

Color Space Support

☐ RGB | YUV 4:4:4 | YUV 4:2:2 | YUV 4:2:0

☐ RGB | YUV 4:4:4 | YUV 4:2:2

☒ RGB | YUV 4:4:4

- T6.2 IPv2.2

Color Matrix (disabled demo window)

Top Level Configuration Options

Color Space Support

☐ RGB | YUV 4:4:4 | YUV 4:2:2 | YUV 4:2:0

☐ RGB | YUV 4:4:4 | YUV 4:2:2

☒ RGB | YUV 4:4:4

la ordenación de canales es **BGR**

ordenación de canales **RGB**

f) AXIS_subset_converter (deshabilitar TSTRB y TKEEP)

- Kria

Slave Interface Signal Properties

Master Interface Signal Properties

Extra Settings

TSTRB Remap String: 1'b0

TKEEP Remap String: 1'b0

- T6.2

Slave Interface Signal Properties

Master Interface Signal Properties

Extra Settings

TSTRB Remap String: 1'b0

TKEEP Remap String: 1'b0

g) pixel_pack_2 (importante: configurar por Sw el modo V_8 #define V_8 2)

- Kria

- T6.2

h) AXI_vdma

- Kria

- T6.2

Basic Advanced

Enable Asynchronous Mode (Auto)

Enable Single AXI4 Data Interface

Enable Vertical Flip

Write Channel Options

Fsync Options: s2mm tuser

GenLock Mode: Dynamic-Master

Read Channel Options

Fsync Options: None

GenLock Mode: Master

Basic Advanced

Enable Asynchronous Mode (Auto)

Enable Single AXI4 Data Interface

Enable Vertical Flip

Write Channel Options

Fsync Options: None

GenLock Mode: Dynamic-Master

Read Channel Options

Fsync Options: None

GenLock Mode: Master

Test 1:

- a) comprobar que se ha utilizado la versión 2 del pixel_packer
- b) comprobar que se escribe en el registro correcto (comprobar dirección)
- c) comprobar que se configuran todos los IPs correctamente desde el Sw

Pruebas:

- d) no cambiar el colorspace y configurar pixel_packer_2 con el parámetro 0, se debería recibir la imagen BGR
- e) no cambiar el colorspace y configurar pixel_packer_2 con el parámetro 2, se debería recibir solo el canal B

Soluciones adoptadas:

- configuración correcta de los registros de V_proc_sys:
 - width, height, clamp_min, clamp_max, start
- configuración correcta de VDMA:
 - Fsync: None
 - Indicar el tamaño de los pixels, originalmente era 4bytes/pix, ahora es 1byte/pixel. Esto se indica en el registro S2MM_FRMDLY_STRIDE

Problema dientes de sierra en los bordes

Original (imagen inferior):

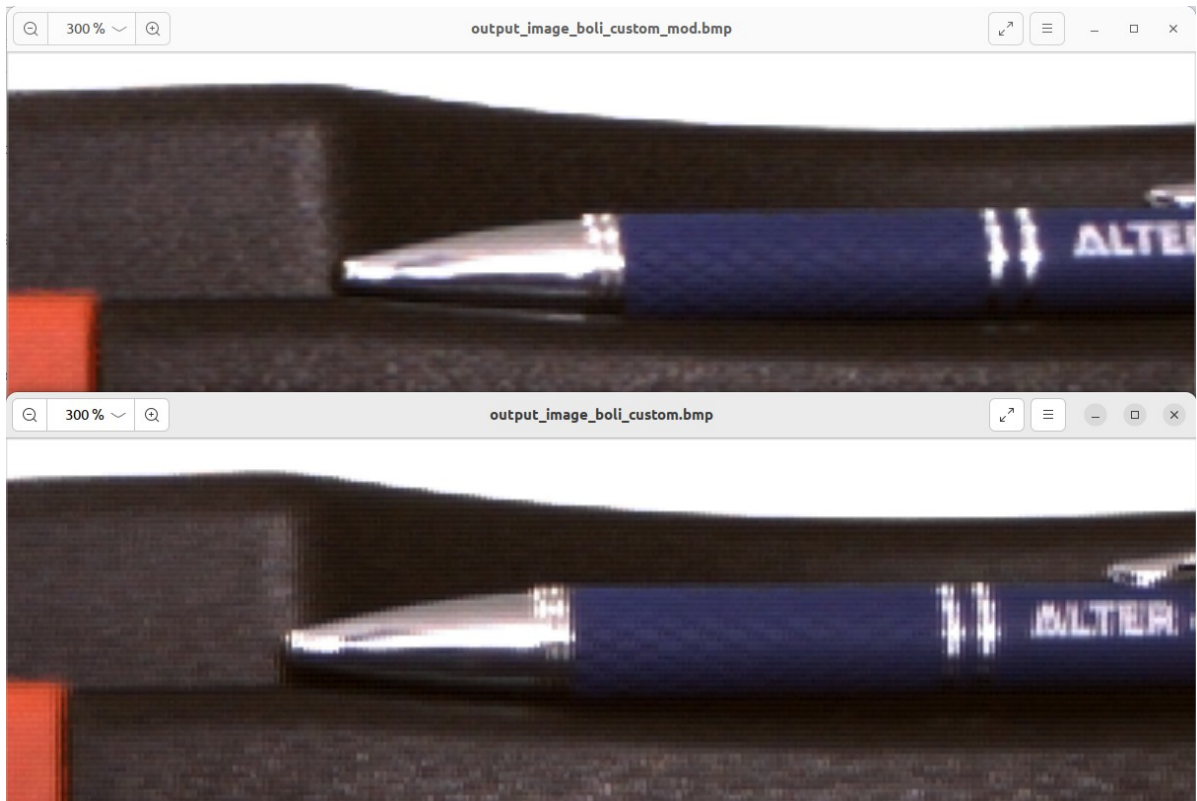
- Input (MSB to LSB): R1 B1 G1 | R0 B0 G0
- the swap is defined as TDATA remap string: tdata[15:0], tdata[23:16], tdata[39:24], tdata[47:40]
- Output (MSB to LSB): B0 G0 R0 | B1 G1 R1

Modificado (imagen superior):

- Input (MSB to LSB): R1 B1 G1 | R0 B0 G0
- tdata[39:24], tdata[47:40], tdata[15:0], tdata[23:16]
- Output (MSB to LSB): B1 G1 R1 | B0 G0 R0

Table: Dual Pixels per Clock, 10 Bits per Component
Mapping for RGB

63:60	59:50	49:40	39:30	29:20	19:10	9:0
zero padding	R1	B1	G1	R0	B0	G0



11. Conclusiones

Para trabajar en niveles de gris con profundidad 8 pixels es necesario modificar parte del pipeline de ImgConditioning:

- (1) Verificar el template que utiliza la cámara (bayern pattern) y los samples per clock del interfaz mipi. En función de esto configurar el demosaic y la ordenación de los canales que debe realizar el **AXIStream Subset Converter** (sin configuración por Sw).
- (2) Incluir **Video Processing Subsystem**: para realizar la conversión de BGR a YCRCB. Modo Color Space Conversion Only y configuración:

Top Level | Color Matrix

Samples Per Clock: 2

Maximum Data Width: 8

Maximum Number of Pixels: 1920 [64 - 8192]

Maximum Number of Lines: 1080 [64 - 4320]

Video Processing Functionality: Color Space Conversion Only

Top Level Configuration Options

Color Space Support

☐ RGB | YUV 4:4:4 | YUV 4:2:2 | YUV 4:2:0

☐ RGB | YUV 4:4:4 | YUV 4:2:2

☒ RGB | YUV 4:4:4

- Los coeficientes de conversión se escriben por sw utilizando los drivers de Xilinx.
Matriz de conversión:

- factor de escala de los coef = 4096
- factor de escala de los offset = 256
- Valor de los coeff **ITU-R BT.601**,

Utilizar la conversión **GRAY = 0.299*R + 0.587*G + 0.114*B**.

Coeficientes de la matriz de conversión (los valores de U y V son irrelevantes) para canales de entrada **en orden RGB**:

K11= 0.299 * scale_coeff
K12= 0.587 * scale_coeff
K13= 0.114 * scale_coeff
K21..K33 = irrelevantes
R_Off= 0 * scale_off
G_Off= 0.5*scale_off (irrelevante)
B_Off= 0.5*scale_off (irrelevante)

IMPORTANTE: tener en cuenta el orden de los canales de entrada (RGB o BGR)

- En el software, además de escribir los coeficientes de conversión, es necesario configurar los registros de Input Video Format 0 (0x10), Output Video Format 1 (0x18) Width (0x20) y Height (0x28) de la imagen y arrancar el core escribiendo en el registro correspondiente.

```
setReg(v_proc_ss_config.dev_base_addr + 0x20, 640);  
setReg(v_proc_ss_config.dev_base_addr + 0x28, 480);  
setReg(v_proc_ss_config.dev_base_addr, 0x81);
```

- (3) Incluir **AXIStream Subset Converter**: para reordenar los canales (TDATA 48b)
TDATA Rempa String: tdata[15:0], tdata[23:16], tdata[39:24], tdata[47:40]BUG .
TDATA Rempa String: tdata[39:24], tdata[47:40], tdata[15:0], tdata[23:16]

Sin configuración por sw.

- (4) Incluir **Pixel_pack (2ppc)**: para empaquetar sólo la componente Y de los pixels. TDATA 48b (2pix RGB/YUV) => TDATA 64b (dependiendo del modo el número de pixels empaquetados serán más o menos, en el caso V_8 son 8pixels de 8bits, copiando el

componente Y). Configurar en modo V_8 (escribir 2 en el registro correspondiente) por sw utilizando escritura directa en registros. Código del ip en:
PYNQ/boards/ip/hls/pixel_pack_2/pixel_pack.cpp

(5) Configurar el VDMA

- (i) Write channel, Fsync: none
- (ii) Allow Unaligned Transfers , no es necesario
- (iii) **IMPORTANTE:** cambiar el stride S2MM a HSIZE*1 (la distancia en bytes entre el inicio de 2 líneas), registro S2MM_FRMDLY_STRIDE

(6) Incluir el reescalado (downscale) en el pipeline de captura, para transformar la imagen de entrada 1440x1080 => 640x480.

Esquema:

v_proc => Y1U1V1Y0U0V0 => Swap => U1V1Y1U0V0Y0 (48b) => yuv2gray_pack
=> Y3Y2Y1Y0 (32b) => rsz_scalern1 => Y3Y2Y1Y0 (32b) => graypixels_pack => Y7Y6Y5Y4
Y3Y2Y1Y0 (64b)

Troubleshooting	
Errores observados	Solución
Todos los pixels a 0x00	Configurar Clamp_min, Clamp_max en VProcSys IP: 0..255
En la imagen GRAY los blancos y negros aparecen como grises	a) Revisar el patrón Bayer configurado en Demosaic IP sea el correspondiente a la cámara. b) Revisar que Gamma IP está configurado para canales de color de 8b o 10b (8b en nuestro caso). c) Revisar los coeff. de conversión (VProcSys IP). GRAY = 0.299*R + 0.587*G + 0.114*B Los canales de entrada están ordenados de MSB a LSB como BGR. En nuestro caso la matriz de coef. debe ser : $ \begin{aligned} K11 &= 0.114 * \text{scale_coeff} \\ K12 &= 0.587 * \text{scale_coeff} \\ K13 &= 0.299 * \text{scale_coeff} \\ K21..K33 &= \text{irrelevantes} \\ R_Off &= 0 * \text{scale_off} \\ G_Off &= 0.5 * \text{scale_off} \\ &(\text{irrelevante}) \\ B_Off &= 0.5 * \text{scale_off} \\ &(\text{irrelevante}) \end{aligned} $ Verificar si la organización de los canales que viene de Gamma es RBG o RGB o BGR
Aparecen líneas negras intercaladas con las líneas de la imagen	Revisar la configuración de VDMA. El stride del canal S2MM debe ser igual a la distancia en bytes entre una línea y otra. En nuestro caso (8b/pixel): Stride=S2MM a HSIZE*1
La imagen presenta un aspecto de “dientes de sierra” en los bordes de las figuras	El AXIS_subset_converter IP tiene un bug y hace el swapping de los pixels (intercambia las columnas dos a dos). Cambiar la organización de los canales que vienen del VProcSys IP. En el AXIS_subset_converter IP: - Input (MSB to LSB): Y1 U1 V1 Y0 U0 V0 tdata[39:24],tdata[47:40],tdata[15:0],tdata[23:16] - Output (MSB to LSB): U1 V1 Y1 U0 V0 Y0
Las imágenes tienen poca luminosidad	Es posible ajustar por Sw (cambiando los coeff.) el brillo y contraste en VProcSys IP